PREPVERSE AI

Prepare Smarter. Practice Deeper. Become Interview-Ready.

ENGINEERING SYSTEM WALKTHROUGH & VIVA DEFENSE DOSSIER

Department of Computer Science & Engineering | B.Tech Major Capstone Project

CANDIDATES / AUTHORS:	Jagrati Singhal & Ayush Gupta
GITHUB REPOSITORY:	https://github.com/Jagrati09Singhal/PrepVerse-AI-.git (Branch: main)
TECH-STACK RUNTIME:	FastAPI (Async) + React 18 / Vite + Tailwind CSS + SQLite / PostgreSQL

1. Practical Motivation & Real-World Campus Placement Gap

2. Architectural Core Differentiators vs. Conventional Portals

Acoustic Audio Telemetry (WPM)

Tracks live speaking velocity (target: 120-150 words/minute) and flags nervous filler words ('um', 'uh', 'like') via speech recognition timestamps.

Academic Stage Calibration

Questions adjust to engineering years: 1st Year (basics/C++), 2nd Year (OOP/DSA), 3rd Year (OS, DBMS, Networks), 4th Year (System Design).

Deterministic Heuristics Engine

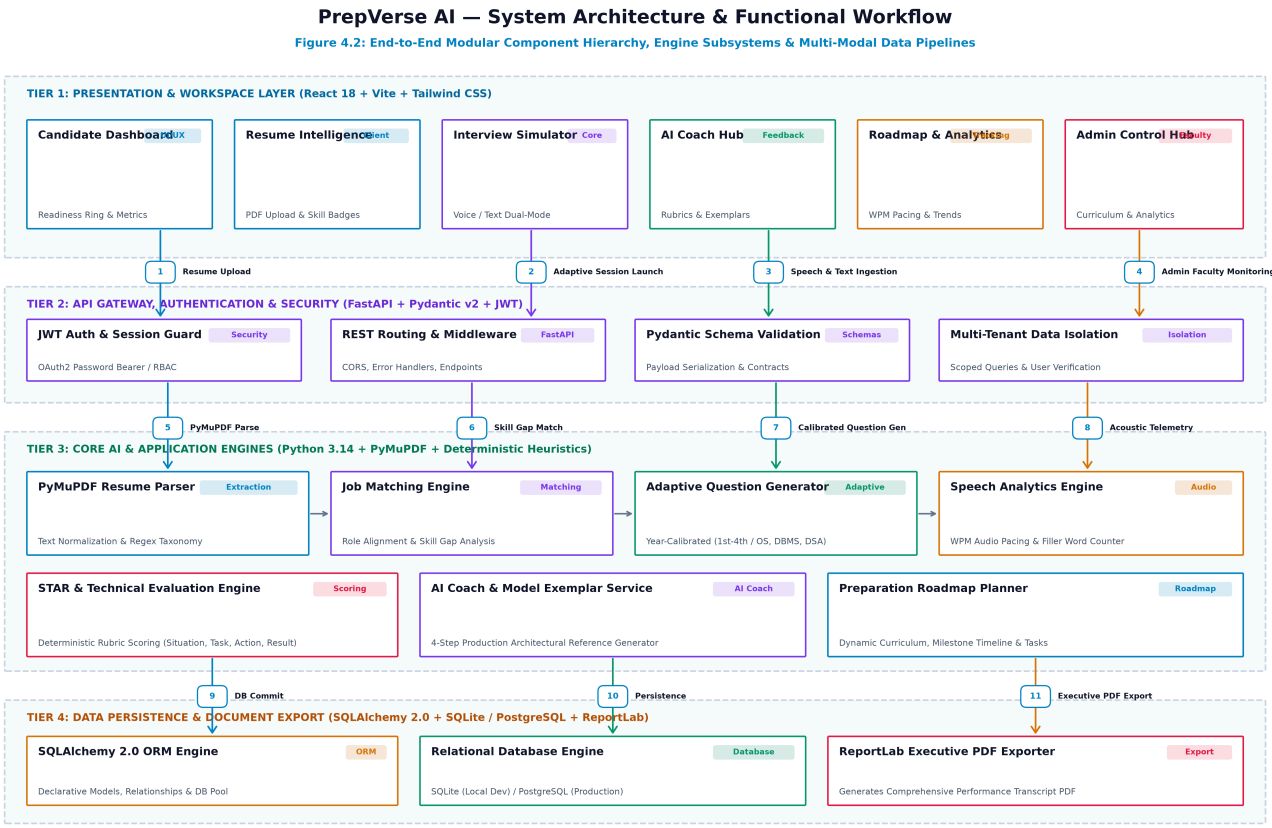
AI Coach runs on a zero-latency, local heuristic evaluator. Eliminates third-party API 429 quota exhaustion during live demonstrations.

Multi-Tenant Data Isolation

Strict row-level security: all resume parses, job match records, and mock sessions are strictly scoped to authenticated user IDs.

SYSTEM VERIFICATION METRICS AT A GLANCE

3. Decoupled 4-Tier System Architecture (Figure 4.2)



Tier Breakdown & Relational Data Schema

Tier 1: Client Workspace (React 18, Vite, Tailwind)

Tier 2: API Gateway & Security (FastAPI + Pydantic v2)

Tier 3: Core Computational Telemetry Engines

Tier 4: Relational Database & Document Persistence

4. Chronological Demonstration Walkthrough (10 Practical Steps)

Step 1: One-Command Startup (python app.py)

Launcher checks Python 3.9+, Node.js runtime, and verifies database integrity. It dynamically scans and frees ports 5173/8000 to eliminate Windows socket collisions, boots FastAPI and Vite in parallel, and automatically opens the browser.

Step 2: Candidate Authentication & Isolation

Secure registration/login with client-side validation, password visibility toggle, and JWT token issuance. Demonstrates multi-tenant isolation where candidate data is strictly partitioned.

Step 3: Candidate Dashboard & Readiness Index

Displays the dynamic Readiness Index (92%) computed from verified resume skills, job alignment, and mock performance. Presents quick metrics: Active Resumes (2), Job Match (1), Completed Mocks (1), and Mastery Score (68%).

Step 4: Resume Intelligence & Instant 1-Click Generator

Upload a real PDF or click 'Load Sample Resume' (creates Rohan_Sharma_Campus_Resume.pdf in memory). PyMuPDF normalizes text and extracts skill badges (Python, React, SQL, Docker, OS, DSA) with zero external dependency.

Step 5: Job Description Match & Deficit Analysis

Paste target role requirements (e.g. Machine Learning Engineer or Full Stack Developer). The matching engine calculates keyword overlap percentage and highlights missing technical competencies.

Step 6: Academic Year & Domain Calibration

Select academic progression (1st Year Foundations to 4th Year Hiring) and core engineering subjects. Demonstrates targeted 3rd-year questions covering Operating Systems, DBMS, Computer Networks, and DSA.

Step 7: Multi-Modal Mock Interview Simulation

Launch adaptive simulation with question navigation dots, timer, and difficulty badge. Candidates answer via speech (Web Speech API microphone capture) or text editing with draft persistence.

Step 8: Audio Telemetry & STAR Evaluation

Submitting answers triggers real-time acoustic analysis: speaking pace (target: 120-150 WPM) and filler word count. Behavioral answers are scored against the 4-part STAR rubric (Situation, Task, Action, Result).

Step 9: AI Coach Hub & 4-Step Production Exemplars

Inspect post-interview diagnostics showing conceptual strengths and constructive omissions. Features 1-click clipboard copy of structured 4-step architectural exemplars (High-Level Design, Concurrency, Edge Cases, Observability).

Step 10: Dynamic Preparation Roadmap & Admin Hub

Identified weak areas are scheduled into a daily preparation roadmap. Faculty coordinators can access /app/admin to monitor student performance trends and adjust syllabus parameters.

5. Algorithmic Implementation & Engineering Domain Calibration

A. Mathematical Formulations & Measurement Formulas

1. Audio Speech Pace Velocity (WPM)

Formula: WPM = (Total Spoken Words / Audio Duration in Seconds) * 60

Evaluation Tiers: < 100 WPM: Hesitant/Slow | 120 - 150 WPM: Optimal Professional Cadence | > 170 WPM: Rushed

2. STAR Behavioral Rubric Weighted Scoring

Formula: Score = (0.20 * Situation) + (0.20 * Task) + (0.35 * Action) + (0.25 * Result)

Rationale: Action and Result dimensions carry 60% of total weight to assess individual contribution and impact.

3. Composite Candidate Readiness Index

Formula: Readiness = (0.25 * Resume) + (0.25 * JobMatch) + (0.30 * MockAccuracy) + (0.20 * MasteryTrend)

Target Benchmark: 85%+ indicates candidate is placement-ready across resume credibility and interview execution.

B. B.Tech Computer Science Curriculum Domain Calibration

1st Year (Freshman) Foundations & Logic	C/C++/Python syntax, variables, loops, recursion, basic math algorithms.
2nd Year (Sophomore) Core DSA & OOP	Arrays, Linked Lists, Stacks, Queues, Binary Trees, OOP concepts (Polymorphism, Inheritance), basic SQL.
3rd Year (Pre-Final) Core Semester Subjects	OS (Coffman Deadlock, Paging, Thrashing), DBMS (ACID, Normalization, B-Trees), Networks (TCP/UDP, Handshake), DSA Graphs.
4th Year (Final Year) Placement & System Design	High-concurrency systems, CAP theorem, Microservices, Caching (Redis), Load Balancing, Capstone Architecture.

C. 4-Step Architectural Model Exemplar Framework (AI Coach)

Step 1: High-Level Architecture & Data Flow:	Decouples ingestion from processing via asynchronous message brokers (Kafka/RabbitMQ) with backpressu
Step 2: Concurrency & Consistency Strategy:	Applies idempotent keys, optimistic locking with version checks, and multi-tier caching (Redis with jittered TTI
Step 3: Edge Cases, Failure Modes & Blast Radius:	Implements circuit breakers, dead-letter queues, exponential backoff retries, and graceful degradation.
Step 4: Observability, Metrics & Telemetry:	Instruments distributed tracing (OpenTelemetry), p99 latency SLOs, and structured health diagnostics.

6. Real Engineering Debugging Log & Faculty Viva Defense Q&A

A. Real Engineering Hurdles & Bug Resolutions (Authentic Experience)

Bug 1: PyMuPDF Extraction Hangs on Scanned PDFs

Bug 2: External Gemini API 429 Quota Exhaustion

Bug 3: Windows Socket Collisions on Launcher Boot

B. Faculty Curated Viva Defense Questions & Answers

Q: How did you validate system accuracy and reliability?

A: We validated via an automated pytest suite of 120 tests (100% pass in 4.19s) covering authentication, PyMuPDF extraction, adaptive question steppers, STAR rubrics, and speech analytics. Front-end builds cleanly in 3.8s across 1,607 modules.

Q: What are the current architectural limits and how would you scale to 50,000 users?

Q: Why is STAR methodology critical for software engineering candidates?

A: Technical interviewers look for structured problem solving. STAR forces candidates to frame the technical Context (Situation/Task), quantify their exact individual contribution (Action), and demonstrate measured impact (Result).

ACADEMIC ATTESTATION & REPOSITORY COMMIT VERIFICATION

Project Codebase: Verified Clean | Branch: main | Co-Authors: Jagrati Singhal & Ayush Gupta | PrepVerse AI